



SJF Scheduling (Preemptive)

Shortest Remaining Time First (SRTF) – the preemptive variant of Shortest Job First

SRTF EXECUTION



P1 IS PREEMPTED AT T=1 AND RESUMES AT T=6 – SAME COLOR, TWO SLICES

4	9	1	1.75
PROCESSES	MAKESPAN (UNITS)	PREEMPTION	AVG. WAITING TIME

COURSE	CSE 402 – Operating System Lab
EXPERIMENT	Preemptive SJF (SRTF) Scheduling Simulation
REPOSITORY	github.com/ashadsetu/CSE-402

SUBMITTED BY

MD Asadullah

Reg. No 23101126 · Section C1
4th Year, 1st Semester

SUBMITTED TO

Atia Rahman Orthi

Lecturer, Department of CSE
University of Asia Pacific

PROBLEM

Schedule four processes with preemptive SJF (Shortest Remaining Time First) — at every time unit, run whichever arrived process has the least remaining burst, preempting the current one if needed.

PID	AT	BT
P1	0	4
P2	1	2
P3	3	2
P4	2	1

Unlike non-preemptive SJF, this version re-checks every single time unit (`remaining_time[i] < shortest`) rather than only at completion, so a process already running can be interrupted the moment something shorter arrives.

■ CODE — SRTF

```
processes = ["P1", "P2", "P3", "P4"]
arrival_time = [0, 1, 3, 2]
burst_time = [4, 2, 2, 1]

n = len(processes)
remaining_time = burst_time.copy()
completion_time = [0] * n
waiting_time = [0] * n
turnaround_time = [0] * n
done = [False] * n

time = 0
completed = 0
gantt = []

while completed < n:
    idx = -1
    shortest = 9999

    for i in range(n):
        if arrival_time[i] <= time and not done[i] and remaining_time[i] > 0:
            if remaining_time[i] < shortest:
                shortest = remaining_time[i]
                idx = i

    if idx == -1:
        time += 1
        continue

    if not gantt or gantt[-1][0] != processes[idx]:
        gantt.append([processes[idx], time, time + 1])
    else:
        gantt[-1][2] = time + 1

    remaining_time[idx] -= 1
    time += 1

    if remaining_time[idx] == 0:
        done[idx] = True
        completed += 1
        completion_time[idx] = time
        turnaround_time[idx] = completion_time[idx] - arrival_time[idx]
        waiting_time[idx] = turnaround_time[idx] - burst_time[idx]

avg_wt = sum(waiting_time) / n
avg_tat = sum(turnaround_time) / n

print("Gantt Chart:")
for g in gantt:
    print(f"{g[0]}({g[1]}-{g[2]})", end=" ")
```

■ RESULT — SRTF

PID	AT	BT	CT	TAT	WT
P1	0	4	9	9	5
P2	1	2	3	2	0
P3	3	2	6	3	1
P4	2	1	4	2	1

```
Gantt Chart: P1(0-1) -> P2(1-3) -> P4(3-4) -> P3(4-6) -> P1(6-9)
Average TAT = 4.0      Average WT = 1.75
```

GANTT CHART

Drawn to scale by duration rather than evenly spaced — P1's two slices make the preemption visible at a glance.



DISCUSSION

- **P1 is preempted almost immediately.** It's the only process ready at time 0, so it starts running — but at $t=1$, P2 arrives with 2 units remaining versus P1's 3, and the scheduler switches instantly. That single-unit check, run every tick instead of only at completion, is what turns SJF into SRTF.
- **The tie at $t=2$ exposes the tie-breaking rule.** When P4 arrives, both P2 and P4 have exactly 1 unit remaining. Because the code only switches on strictly-less-than (`remaining_time[i] < shortest`) and scans in index order, P2 keeps running rather than being swapped out — the loop's first match wins ties, not the newest arrival. A different tie-break rule could have produced a different, equally valid schedule.
- **P1 pays for being first and largest.** It waits 5 of its 9 total turnaround units despite arriving before anyone else — every other process cuts in line because each one is shorter. P2, P3, and P4 each wait at most 1 unit. That's the same trade-off non-preemptive SJF makes, just enforced more aggressively: SRTF will interrupt work already in progress, not only work waiting to start.
- **Preemption isn't free in practice.** This simulation charges nothing for switching processes, but a real OS pays a context-switch cost — saving and restoring registers, flushing caches — every time it swaps. One preemption here is cheap; a workload that preempts constantly could lose more to overhead than it gains in responsiveness.

CONCLUSION

- SRTF produced an average turnaround of 4.0 and average wait of 1.75 for this workload — both are at least as good as any non-preemptive ordering could achieve, since SRTF is provably optimal for minimizing average waiting time when arrival times differ.
- **The gain comes directly from interrupting P1** — letting three shorter jobs finish first shrinks their individual waits to nearly zero, at the cost of stretching P1's.
- That's the same throughput-versus-fairness trade as non-preemptive SJF, just sharper: SRTF reacts the instant a shorter job appears instead of waiting for the current one to finish.

SRTF minimizes average wait on this workload (1.75) by preempting **P1** once — the same process ends up carrying the most wait (5 units) of any process here.